

isthmus-ablation-core Manual

Voxel ablation and coupling core

June 9, 2026

Contents

1	isthmus-ablation-core	2
1.1	Quick Start	2
1.2	Current Capabilities	2
1.3	Rebuild Docs	3
1.4	Current Run Type	3
1.5	First Example	3
2	Getting Started	5
2.1	One-Time Shell Setup	5
2.2	Build DSMC/IAC	5
2.3	Run A DSMC/IAC Case	6
2.4	Standalone Only	6
2.5	Visual Outputs	6
2.6	Rebuild The Manual	6
3	Architecture	7
3.1	ISTHMUS Coupling	8
3.2	Future DSMC/SPARTA Coupling	8
4	Verification	9
4.1	Norms	9
4.2	Convergence	9
5	Expected Features	10
5.1	Voxel Core	10
5.2	Ablation Control	10
5.3	ISTHMUS Coupling	10
5.4	DSMC/SPARTA Coupling	11
6	Command Reference	12

7	include Command	13
7.1	Syntax	13
7.2	Examples	13
7.3	Description	13
8	voxel Command	14
8.1	Syntax	14
8.2	Examples	14
8.3	Description	14
8.4	Current Limits	14
8.5	Planned Extensions	14
9	source Command	16
9.1	Syntax	16
9.2	Example	16
9.3	Description	16
9.4	Current Limits	16
9.5	Planned Extensions	16
10	timestep Command	17
10.1	Syntax	17
10.2	Examples	17
10.3	Description	17
11	Loop Commands	18
11.1	Syntax	18
11.2	Example	18
11.3	Description	18
12	voxel ablate Command	19
12.1	Syntax	19
12.2	Example	19
12.3	Description	19
12.4	Current Limits	19
13	fix ... voxel/ablate Command	20
13.1	Syntax	20
13.2	Example	20
13.3	Description	20
13.4	Current Limits	20
13.5	Planned Extensions	20

14	isthmus surface Command	21
14.1	Syntax	21
14.2	Example	21
14.3	Description	21
14.4	DSMC Coupling Note	21
15	surface Commands	22
15.1	Syntax	22
15.2	Examples	22
15.3	Description	23
15.4	Current Limits	24
16	stats and stats_style Commands	26
16.1	Syntax	26
16.2	Example	26
16.3	Description	26
16.4	Current Columns	26
16.5	DSMC/SPARTA Note	26
17	voxel dump Command	27
17.1	Syntax	27
17.2	Examples	27
17.3	Description	27
17.4	History Columns	28
17.5	VTU Cell Data	28
17.6	Planned Extensions	28
18	verify Command	29
18.1	Syntax	29
18.2	Examples	29
18.3	Description	29
18.4	Convergence	29
18.5	Norms	30
18.6	Expression Variables	30
19	run Command	31
19.1	Syntax	31
19.2	Examples	31
19.3	Description	31
20	Direct Slab Ablation	32

21 ISTHMUS Slab Ablation	34
22 Sphere ISTHMUS Ablation	35
22.1 Inputs	35
22.2 Run	35
22.3 Outputs	35
22.4 Verification	35
23 DSMC Sphere Kinetic Example	36
23.1 Chemistry Loop	36
23.2 Analytical Comparison	37
23.3 Visualization Case	38
24 Directory Layout	39
25 Build And Link	40
25.1 What Is Built	40
25.2 Root Discovery	40
25.3 Recommended DSMC/IAC Build	41
25.4 DSMC Machine Target	41
25.5 How The Overlay Works	41
25.6 Standalone Build	42
25.7 Debugging The Overlay	42
26 Testing	44
26.1 Test Organization	45
27 Test Reports	47
27.1 Command-Line Report Output	47
27.2 Report Cases	47
27.3 Build The Combined Report	47
27.4 Direct Tool Use	48
27.5 Convergence Reports	48
28 Documentation	49
28.1 Source Format	49
28.2 PDF Manual	49
28.3 Update Rule	50

1 isthmus-ablation-core

isthmus-ablation-core is a C++ voxel ablation and coupling core. It owns solid voxel state, tracks mass and material properties, supports standalone verification runs, and is being shaped so DSMC/SPARTA can call it through thin commands, fixes, and computes.

The project is strongly linked to ISTHMUS, but it is not only an ISTHMUS wrapper. ISTHMUS is the surface reconstruction and surface-to-voxel mapping backend. The core should also remain useful for voxel bookkeeping, local voxel-face ablation, generated geometries, and exact-solution verification without a fluid domain.

1.1 Quick Start

If DSMC and ISTHMUS are already installed, set their roots once:

```
export DSMC_ROOT=$HOME/dsmc
export ISTHMUS_ROOT=$HOME/isthmus
```

Then build a private DSMC/IAC executable from this repository:

```
cmake --preset dsmc
cmake --build --preset dsmc
ctest --preset dsmc
```

The executable is:

```
build-dsmc/bin/dsmc-iac
```

This does not edit the DSMC checkout. The build creates a disposable overlay in `build-dsmc/dsmc-overlay/`, where DSMC source files and IAC bridge commands are symlinked together for compilation.

Run a coupled example:

```
build-dsmc/bin/dsmc-iac \
  -in examples/dsmc-sphere-kinetic/in.dsmc-sphere-kinetic
```

For standalone-only work:

```
cmake --preset standalone
cmake --build --preset standalone
ctest --preset standalone
```

1.2 Current Capabilities

- Create a slab voxel model.
- Assign material density.
- Define a constant mass-loss source.
- Use explicit or mass-Courant timesteps.
- Apply local voxel ablation with delete-empty behavior.
- Use simple standalone loops with `variable`, `label`, `next`, and `jump`.
- Print aligned standalone stats with `stats` and `stats_style`.

- Write a CSV history file.
- Write VTU voxel files for visual inspection.
- Compute single-species ideal-gas kinetic-theory flux on ISTHMUS triangles.
- Build a private DSMC executable that calls the core through thin

command-style bridge files without modifying the DSMC checkout.

- Include shared input files with `include`.
- Verify tracked quantities against input-file exact expressions.
- Optionally build visual verification reports from test data.

1.3 Rebuild Docs

Documentation source lives in `docs/`. After changing command syntax, examples, concepts, or behavior, rebuild the single PDF manual:

```
cmake -S . -B build
cmake --build build --target docs-pdf
```

The generated manual is:

```
docs/isthmus-ablation-core-manual.pdf
```

You can also call the local PDF builder directly:

```
python3 tools/build-docs-pdf.py
```

1.4 Current Run Type

The standalone executable currently prints:

```
# Standalone voxel ablation
```

Future ISTHMUS-backed runs should use a more specific title, such as:

```
# Coupled voxel/ISTHMUS surface ablation
```

1.5 First Example

```
units si

voxel material carbon density 1800.0
voxel create solid slab nx 8 ny 4 nz 4 dx 1.0e-6 material carbon

source q1 constant 1.8 units kg/m2/s
timestep mass/courant 0.5 source q1

stats 1
stats_style step time active-voxels deleted-voxels remaining-mass mass-fraction front
```

```
voxel dump hist solid history 1 output/slab-direct-ablation/history.csv
voxel dump vox solid vtu 1 output/slab-direct-ablation/voxels_*.vtu select active scalar mass-fraction

variable i loop 8
label ablate-loop
voxel ablate solid source q1 policy local face xlo delete yes
run 1
next i
jump SELF ablate-loop
```

The regression wrapper in `tests/inputs/slab-direct-ablation/in.slab-direct-command.verify` includes this example and adds exact-solution checks with `verify`.

2 Getting Started

The easiest DSMC/IAC workflow is:

- Install or build DSMC and ISTHMUS somewhere on your machine.
- Tell this repository where they are.
- Build a private DSMC executable inside this repository's build directory.

Your DSMC checkout is not modified. The build creates an overlay under `build-dsmc/dsmc-overlay/` and writes the executable to:

```
build-dsmc/bin/dsmc-iac
```

2.1 One-Time Shell Setup

If DSMC and ISTHMUS live in stable locations, put these in `~/.bashrc`, `~/.zshrc`, or your shell startup file:

```
export DSMC_ROOT=$HOME/dsmc
export ISTHMUS_ROOT=$HOME/isthmus
```

Then open a new terminal or source the file:

```
source ~/.zshrc
```

You can skip this and pass paths directly to CMake instead:

```
cmake --preset dsmc -DDSMC_ROOT=/path/to/dsmc -DISTHMUS_ROOT=/path/to/isthmus
```

2.2 Build DSMC/IAC

From a fresh clone:

```
cmake --preset dsmc
cmake --build --preset dsmc
ctest --preset dsmc
```

The `dsmc` build preset:

- builds `libisthmus_ablation_core.a`;
- creates `build-dsmc/dsmc-overlay/src`;
- symlinks DSMC source files into that overlay;
- symlinks the IAC bridge command files into that overlay;
- generates the overlay `Makefile.package`;
- runs DSMC's normal `make <machine>` build there;
- writes `build-dsmc/bin/dsmc-iac`.

The default DSMC machine target is `mac_mpi`. Override it with:

```
cmake --preset dsmc -DDSMC_MACHINE=mpi
```


2.3 Run A DSMC/IAC Case

```
build-dsmc/bin/dsmc-iac \  
-in examples/dsmc-sphere-kinetic/in.dsmc-sphere-kinetic
```

Run the fast DSMC verification suite:

```
ctest --preset dsmc
```

2.4 Standalone Only

The standalone binary does not need DSMC:

```
cmake --preset standalone  
cmake --build --preset standalone  
ctest --preset standalone
```

Run the first standalone example:

```
./build/ia-core -in examples/slab-direct-ablation/in.slab-direct-ablation
```

2.5 Visual Outputs

Examples write under `output/` by default. For the direct slab example:

```
output/slab-direct-ablation/history.csv  
output/slab-direct-ablation/voxels_000000.vtu  
output/slab-direct-ablation/voxels_000001.vtu  
...
```

Open VTU/VTP files directly in ParaView. The examples usually write active voxels only and select `mass-fraction` as the first visible cell scalar.

2.6 Rebuild The Manual

```
cmake --build --preset docs
```

The PDF is written to:

```
docs/isthmus-ablation-core-manual.pdf
```

3 Architecture

The project has two intended front doors into one shared core:

- Standalone mode: this repository owns parsing, timestep advancement, stats,

dumps, and verification.

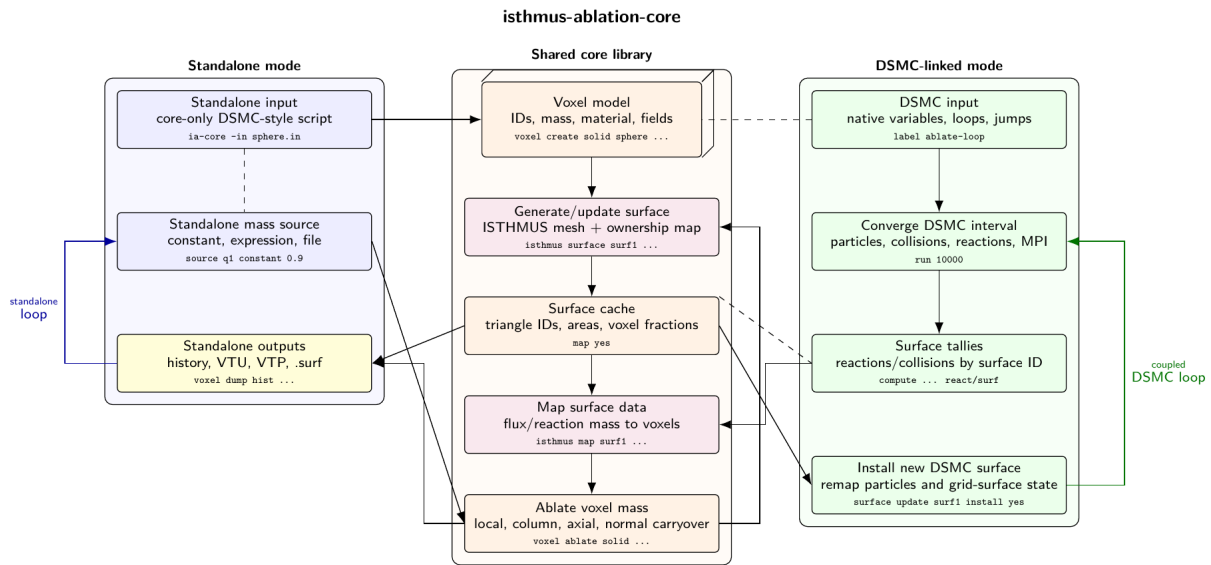
- DSMC/SPARTA-linked mode: DSMC/SPARTA owns the main input parser and timestep

loop, while this project contributes commands, computes, and a core library object.

The shared core owns:

- voxel state with stable IDs;
- material density and voxel mass;
- generated and imported geometries;
- mass-loss sources;
- ablation policies;
- diagnostics and history;
- optional ISTHMUS surface state and ownership mapping.

The current flowchart is available here:



Architecture flowchart

Architecture flowchart

3.1 ISTHMUS Coupling

The current standalone ISTHMUS path is:

- Convert active voxels to an `isthmus::VoxelSet`.
- Call `isthmus::MarchingWindows`.
- Cache the surface mesh and triangle-to-voxel ownership fractions.
- Run `surface flux ...` to apply mass flux to selected triangles.
- Run `voxel ablate ...` to update voxel mass and delete empty voxels.
- Run `isthmus surface ...` again if the voxel state changed.

3.2 Future DSMC/SPARTA Coupling

The planned DSMC coupling should support both a command-loop path and a timestep-callback path. We will implement whichever proves easiest first.

The command-loop path would look like:

```
label ablate-loop
run 10000
surface flux surf1 source dsmc-flux select all
voxel ablate solid surface surf1 policy local delete yes
isthmus surface surf1 voxels solid map yes
surface update surf1 install yes
jump SELF ablate-loop
```

The loop keeps DSMC/SPARTA in charge of particles, reactions, variables, labels, jumps, MPI, and stats. This project supplies commands that operate on the shared voxel/surface core between DSMC run intervals.

The callback path would expose similar core operations through a fix-style or other timestep callback so geometry can update during a longer run. That may be more natural for cases that need frequent ablation updates.

The main tradeoff is timing control. Command loops are explicit and fit native DSMC/SPARTA `label`, `jump`, and chunked `run` workflows. Callback coupling can hide more machinery behind a timestep hook, but may be more convenient for fine-grained updates.

4 Verification

Verification should be input-driven, not hardcoded in the core model.

The core tracks actual quantities such as:

- `remaining-mass`
- `mass-fraction`
- `volume-fraction`
- `front`
- `active-voxels`
- `deleted-voxels`

The input file declares exact or reference expressions:

```
verify mass-fraction exact "1.0 - q1*time/(rho*length)" tolerance 0.01 percent norm max
verify volume-fraction exact "1.0 - q1*time/(rho*length)" tolerance 0.01 percent norm final
verify front exact "q1*time/rho" tolerance 0.01 percent norm final
verify radius exact "initial-radius - q1*time/rho" tolerance 3.0 percent norm final
```

This keeps geometry-specific exact solutions out of the source code.

`mass-fraction` is the smooth remaining-mass ledger normalized by initial mass. `volume-fraction` is the active voxel count normalized by the initial active voxel count. It changes only when voxels are deleted, so it captures the stair-stepped voxelized geometry response.

4.1 Norms

`norm final` checks only the last history row.

`norm max` checks the maximum absolute error over recorded history rows.

`norm rms` checks the root-mean-square error over recorded history rows.

Regression tests should prefer percent tolerances unless an absolute error scale is more meaningful.

4.2 Convergence

Convergence checks are input-driven too. A test can define variables, use them inside the case setup, and ask the executable to rerun the same input with overrides:

```
variable resolution equal 20
variable steps equal 4
voxel create solid sphere diameter 1.0e-3 resolution ${resolution} material carbon
variable i loop ${steps}
```

```
convergence radius exact "initial-radius - q1*time/rho" tolerance 100.0 percent norm final vary resolution 5 10 20 va
convergence volume-fraction exact "((initial-radius - q1*time/rho)/initial-radius)^3" tolerance 100.0 percent norm fi
```

This keeps convergence criteria in the test input instead of a separate helper script.

5 Expected Features

This page tracks planned user-visible capabilities. It is intentionally more stable than planning notes and brainstorm documents.

5.1 Voxel Core

- Generate slab, sphere, cylinder/cone, and imported voxel geometries.
- Track stable voxel IDs, lattice coordinates, material, remaining mass,

activity, fixed state, and future fields.

- Support local ablation, column carryover, axial carryover, normal-directed

carryover, ghost voxels, and configurable boundary behavior.

- Dump history, VTU voxel state, restart state, and diagnostic summaries.

5.2 Ablation Control

The long-term design should support both explicit commands and timestep callbacks. We will implement whichever path is easiest first while keeping the core API usable by both.

The command-oriented form is useful for readable standalone scripts and DSMC/SPARTA loops:

```
label ablate-loop
voxel ablate solid source q1 policy local face xlo delete yes
isthmus surface surf1 voxels solid map yes
surface flux surf1 source dsmc-tallies select all
voxel ablate solid surface surf1 policy local delete yes
jump SELF ablate-loop
```

Standalone inputs can loop over these commands directly. DSMC/SPARTA inputs can use native variables, labels, jumps, and repeated `run` chunks to couple gas evolution and voxel/surface updates.

The callback form may use a fix-style or other timestep hook to call the same core operations during a longer DSMC/SPARTA run.

5.3 ISTHMUS Coupling

- Generate a surface from active voxels using ISTHMUS.
- Cache triangle IDs, areas, normals, and triangle-to-voxel ownership

fractions.

- Map surface quantities back to voxel mass loss.
- Regenerate the surface when voxel state changes.
- Export VTP and SPARTA `.surf` files in standalone mode.
- Install or replace explicit DSMC/SPARTA surfaces in coupled mode.

5.4 DSMC/SPARTA Coupling

- Let DSMC/SPARTA own particle advancement, collisions, reactions, MPI,

variables, loops, and stats.

- Read per-surface DSMC tallies after a `run` interval.
- Map those tallies to ISTHMUS triangles and then to voxels.
- Apply voxel mass loss with chosen policies.
- Regenerate and reinstall the surface before the next `run` interval.

6 Command Reference

The command language is intentionally close to DSMC/SPARTA style: one command per line, simple positional arguments, and keyword/value options.

New command names avoid underscores where this project controls the syntax. Existing DSMC/SPARTA commands keep their native spelling, for example `stats_style` and `surf_modify`.

Current standalone commands:

`include` (include.md)

`voxel` (voxel.md)

`source` (source.md)

`timestep` (timestep.md)

`el`, `next`, and `jump` (loops.md)

`voxel ablate` (voxel-ablate.md)

`.. voxel/ablate` (fix-voxel-ablate.md)

`isthmus surface` (isthmus-surface.md)

`surface` (surface.md)

`iac` (iac.md) for DSMC-hosted diagnostics and verification

and `stats_style` (stats.md)

`voxel dump` (voxel-dump.md)

`verify` (verify.md)

`run` (run.md)

7 include Command

The `include` command reads another input file at the point where the command appears. This keeps examples and tests from duplicating the same setup.

7.1 Syntax

```
include <path>
```

7.2 Examples

```
include ../../../../examples/slab-direct-ablation/in.slab-direct-ablation
```

7.3 Description

Relative paths are resolved from the directory of the file that contains the `include` command.

The command is useful for regression tests:

```
include ../../../../examples/slab-direct-ablation/in.slab-direct-ablation

verify remaining-mass exact "initial-mass - q1*area*time" tolerance 0.01 percent norm max
verify mass-fraction exact "1.0 - q1*time/(rho*length)" tolerance 0.01 percent norm max
verify front exact "q1*time/rho" tolerance 0.01 percent norm final
```

The example owns the physical case. The test wrapper owns the pass/fail criteria.

8 voxel Command

The `voxel` command family creates and configures voxel-state objects.

8.1 Syntax

```
voxel material <name> density <rho>
voxel create <model> slab nx <nx> ny <ny> nz <nz> dx <dx> material <name>
voxel create <model> sphere diameter <D> dx <dx> material <name>
voxel create <model> sphere diameter <D> resolution <N> material <name>
voxel write-history <model> <path>
```

8.2 Examples

```
voxel material carbon density 1800.0
voxel create solid slab nx 8 ny 4 nz 4 dx 1.0e-6 material carbon
voxel create solid sphere diameter 1.0e-3 dx 5.0e-5 material carbon
voxel create solid sphere diameter 1.0e-3 resolution 20 material carbon
voxel ghost solid axis y boundary infinite layers 1
voxel write-history solid output/dsmc-sphere-kinetic/history.csv
```

8.3 Description

`voxel material` defines a material and its density.

`voxel create` creates a named voxel model. Each voxel receives a stable ID, integer lattice indices, a centroid, an initial mass based on density and dx^3 , and an active flag.

`slab` fills the full rectangular lattice.

`sphere` fills voxel centroids inside a sphere with diameter `D`. Specify either `dx` or `resolution`, but not both. `resolution <N>` means `N` voxels across the sphere diameter, so $dx = D/N$. Voxels outside the sphere remain in the bounding lattice as inactive cells so the structured indexing and voxel IDs stay stable while the sphere ablates.

`voxel ghost` configures ghost voxel images used during surface generation. The first supported boundary is `boundary infinite`, which extrapolates active boundary voxels beyond both ends of the requested axis. Ghost voxels do not carry independent mass; their surface ownership maps back to the corresponding real voxel. This is useful for making a finite slab patch behave like an infinite wall during ISTHMUS marching cubes.

`voxel write-history` writes the current in-memory history table immediately. Standalone inputs usually use `voxel dump <id> <model> history ...`; the direct write command is mainly for DSMC-hosted scripts, where DSMC owns the loop and the core history should be flushed after the coupled loop completes.

8.4 Current Limits

- Only one voxel model is currently supported.
- Only one material is currently supported.
- Slab and centroid-filled sphere geometry are currently implemented.
- Ghost voxels currently support `boundary infinite`.

8.5 Planned Extensions

- TIFF import.

- CSV import.
- Periodic and open ghost voxel behavior.

9 source Command

The **source** command defines mass-loss data. It is available in standalone inputs and in DSMC-hosted inputs through the bridge.

9.1 Syntax

```
source <id> constant <value> units <units>
```

9.2 Example

```
source q1 constant 1.8 units kg/m2/s
```

9.3 Description

The current implementation supports a constant mass flux source. The value is interpreted as mass flux in **kg/m2/s** for the local slab ablation case.

In DSMC-hosted inputs, **source** is mainly useful for no-gas verification cases, debug probes, and prescribed ablation studies. Coupled DSMC chemistry cases usually use **surface flux dsmc/reaction** or **surface flux dsmc/surf** instead.

9.4 Current Limits

- Only **constant** sources are implemented.
- Units are recorded but not converted.

9.5 Planned Extensions

- Expressions.
- File or table sources.
- Surface-triangle sources.

10 timestep Command

The `timestep` command sets the standalone time increment.

10.1 Syntax

```
timestep <dt>
timestep mass/courant <C> source <source-id>
```

10.2 Examples

```
timestep 1.0e-6
timestep mass/courant 0.5 source q1
```

10.3 Description

The explicit form sets `dt` directly.

The mass-Courant form chooses:

$$dt = C * \rho * dx / source$$

For the current slab case, $C = 0.5$ removes half of one voxel mass from each exposed column per step.

11 Loop Commands

The standalone runner supports a small DSMC/SPARTA-style loop subset.

11.1 Syntax

```
variable <name> loop <N>
label <name>
next <name>
jump SELF <label>
```

11.2 Example

```
variable i loop 8
label ablate-loop
voxel ablate solid source q1 policy local face xlo delete yes
run 1
next i
jump SELF ablate-loop
```

11.3 Description

`variable <name> loop <N>` creates a loop counter.

`label <name>` marks a target location.

`next <name>` advances the loop counter. When the counter is exhausted, the next `jump` command is skipped.

`jump SELF <label>` jumps to a label in the current input.

The current implementation supports only this compact loop pattern. It is meant to make standalone inputs resemble future DSMC/SPARTA coupled scripts without reimplementing the full DSMC/SPARTA variable language.

12 voxel ablate Command

The `voxel ablate` command applies one timestep of mass loss to a voxel model.

12.1 Syntax

```
voxel ablate <model> source <source-id> policy <policy> face <face> delete <yes|no>
voxel ablate <model> surface <surface-id> policy <policy> delete <yes|no>
```

12.2 Example

```
voxel ablate solid source q1 policy local face xlo delete yes
voxel ablate solid source q1 policy local face yhi delete yes
voxel ablate solid surface skin policy local delete yes
voxel ablate solid surface skin policy carryover/normal delete yes
```

12.3 Description

The command applies one mass-loss update over the current timestep.

With `source`, `policy local` removes mass from the first active voxel in each slab column on the selected face. The required `face` argument accepts `xlo`, `xhi`, `ylo`, `yhi`, `zlo`, or `zhi`.

With `surface`, the command consumes mass already assigned to triangles by `surface flux`, uses the ISTHMUS triangle-to-voxel ownership map, and subtracts that mass from the associated voxels.

`policy local` removes only the mass available in each mapped voxel and records any overshoot as dropped mass.

`policy carryover/normal` conserves overshoot by pushing excess mass loss from a depleted voxel into live 26-neighbor voxels along the inward normal direction. This policy is intended for closed ISTHMUS surfaces such as spheres.

Use `run 1` after `voxel ablate` to advance time, record history, print stats, and write scheduled dumps:

```
voxel ablate solid source q1 policy local face xlo delete yes
voxel ablate solid source q1 policy local face zhi delete yes
run 1
```

A surface-coupled loop is:

```
isthmus surface skin voxels solid buffer 1 weighting no map yes
surface flux skin source q1 select all
voxel ablate solid surface skin policy carryover/normal delete yes
run 1
```

`delete yes` is the default. When a voxel's mass is depleted, it is marked inactive and omitted from VTU dumps that use `select active`.

12.4 Current Limits

- Only one voxel model is currently supported.
- `policy carryover/normal` is currently implemented for surface-backed

ablation.

- The current source must be a constant mass flux.

13 fix ... voxel/ablate Command

The `voxel/ablate` fix applies mass loss to a voxel model during standalone time advancement. It is the compact callback-style alternative to explicit `voxel ablate` commands.

13.1 Syntax

```
fix <id> all voxel/ablate <Nevery> voxels <model> source <source-id> policy <policy> face <face> delete <yes|no>
```

13.2 Example

```
fix ab all voxel/ablate 1 voxels solid source q1 policy local face xlo delete yes
fix ab all voxel/ablate 1 voxels solid source q1 policy local face yhi delete yes
```

13.3 Description

Every `Nevery` steps, the fix applies the selected source to the selected voxel model. The current implementation supports a local slab face update:

- find the first active voxel from the selected face in each slab column;
- remove `source * dx^2 * dt` from that voxel;
- delete the voxel if its mass reaches zero and `delete yes` is set.

The required `face` argument accepts `xlo`, `xhi`, `ylo`, `yhi`, `zlo`, or `zhi`.

`delete yes` is the default behavior. When a voxel's mass is depleted, it is marked inactive and omitted from VTU dumps that use `select active`. Use `delete no` only for cases where depleted voxels should remain in the active voxel set.

13.4 Current Limits

- Only group `all` is accepted.
- Only policy `local` is implemented.
- Only slab face ablation is implemented.

13.5 Planned Extensions

- Column carryover.
- Axial carryover.
- Normal-directed carryover.
- Shared core support for both explicit `voxel ablate` commands and fix-style

timestep callbacks.

14 isthmus surface Command

The `isthmus surface` command reconstructs a triangle surface from active voxels.

14.1 Syntax

```
isthmus surface <surface-id> voxels <model> [buffer <N>] [weighting yes|no] [map yes|no] [crop real|no]
```

14.2 Example

```
isthmus surface skin voxels solid buffer 1 weighting no map yes
isthmus surface skin voxels solid buffer 1 weighting no map yes crop real
```

14.3 Description

The command sends the active voxels in `<model>` to ISTHMUS, runs marching cubes, stores the resulting triangle mesh, and optionally stores triangle-to-voxel ownership fractions.

`map yes` is required when the surface will be used for ablation, because `voxel ablate <model> surface <surface-id>` needs those ownership fractions to send triangle mass loss back to voxels.

`buffer` adds empty voxel layers around the active voxel bounding box before marching cubes. This helps ISTHMUS generate a closed surface as the solid shrinks.

`crop real` removes triangles whose centroids are outside the real voxel domain after ghost voxels are added. Use it when ghost voxels are only a boundary condition for surface quality and flux should still be applied only on the real DSMC/voxel domain.

14.4 DSMC Coupling Note

This command is intentionally an explicit input-file command rather than a hidden fix callback. A future DSMC loop can call it after voxel mass changes and before refreshing the DSMC surface representation.

15 surface Commands

The `surface` command family works with triangle surfaces generated from voxels. These commands are deliberately separate from `voxel` commands because future DSMC-coupled runs will need to refresh surfaces, apply DSMC surface tallies, and then map that information back to voxels inside input-file loops.

15.1 Syntax

```
surface flux <surface-id> source <source-id> select all
surface flux <surface-id> source <source-id> select normal nx <x> ny <y> nz <z> [min-cos <c>]
surface flux <surface-id> source <source-id> select voxels voxels <model>
surface flux <surface-id> kinetic/theory pressure <p> temperature <T> \
  mole-fraction <x> molecular-mass <kg> reaction-prob <alpha> \
  solid-mass-per-hit <kg> select <selector>
surface flux <surface-id> dsmc/surf fix <fix-id> column <N> \
  reaction-prob <alpha> solid-mass-per-hit <kg> \
  [ablation-dt <dt> | mass-courant <C>]
surface flux <surface-id> dsmc/reaction fix <fix-id> column <N> \
  sample-steps <Nstep> solid-mass-per-reaction <kg> \
  [ablation-dt <dt>] [time-scale <S>]
surface measure-flux <surface-id> dsmc/reaction fix <fix-id> column <N> \
  sample-steps <Nstep> expected kinetic/theory number-density <n> \
  mole-fraction <x> temperature <T> molecular-mass <kg> \
  [reaction-prob <alpha>] [solid-mass-per-reaction <kg>]

surface dump <dump-id> <surface-id> vtp <N> <path>
surface dump off

# DSMC bridge only:
surface write-vtp <path> <surface-id> [fix <fix-id> fields <name1> ...]
```

15.2 Examples

```
surface flux skin source q1 select all
surface flux skin source q1 select normal nx 1.0 ny 0.0 nz 0.0 min-cos 0.5
surface flux skin source q1 select voxels voxels solid
surface flux skin kinetic/theory pressure 50.0 temperature 5000.0 \
  mole-fraction 0.21 molecular-mass 5.313e-26 reaction-prob 1.0 \
  solid-mass-per-hit 1.3011869411625376e-23 select all
surface flux skin dsmc/surf fix sflux column 1 reaction-prob 1.0 \
  solid-mass-per-hit 1.99447348e-26 mass-courant 0.25
surface flux skin dsmc/reaction fix rco column 1 sample-steps 20 \
  solid-mass-per-reaction 3.98894696e-26 time-scale 1500
surface measure-flux skin dsmc/reaction fix rco column 1 sample-steps 1 \
  expected kinetic/theory number-density 7.244e23 mole-fraction 0.21 \
  temperature 5000.0 molecular-mass 5.31352e-26 reaction-prob 1.0 \
  solid-mass-per-reaction 3.98894696e-26

surface dump skin skin vtp 10 output/sphere/surface_*.vtp
surface dump off

surface write-vtp skin output/surface-*.vtp \
  fix sqty fields collision-count incident-number-flux pressure
```

15.3 Description

`surface flux` assigns one timestep of mass loss to selected triangles on an existing surface. Source and kinetic-theory modes require an explicit `select all`, `select normal`, or `select voxels` clause. With `source`, the source is a constant mass flux in kg/m2/s. The requested mass on each selected triangle is:

```
mass = flux * triangle-area * timestep
```

With `kinetic/theory`, the command computes the mass flux from ideal-gas impingement theory for one reactive species:

```
Gamma = mole-fraction * pressure/(kB*temperature)
        * sqrt(kB*temperature/(2*pi*molecular-mass))
flux = reaction-prob * solid-mass-per-hit * Gamma
```

This is intended as the first DSMC-coupled verification source: DSMC can run a spatially uniform, stationary hot gas domain while this command applies the corresponding continuum kinetic-theory flux to the current ISTHMUS triangles.

With `dsmc/surf`, the command reads per-surface data from a DSMC `fix ave/surf` instance. The selected column is interpreted as an incident number flux. The bridge converts it to solid mass flux with:

```
flux = number-flux * reaction-prob * solid-mass-per-hit
```

This is the first true DSMC-coupled source path. DSMC owns the gas domain, surface collisions, surface averaging, loops, and remapping commands; this library owns the voxel mass ledger, ISTHMUS surface ownership map, and ablation. The current bridge implementation supports this source on one MPI rank while the command and data path are kept compatible with later distributed storage.

With `dsmc/reaction`, the command reads per-surface reaction counts from a DSMC `fix ave/surf`, usually one that averages `compute react/surf`. The selected column is interpreted as an averaged number of simulation-particle reactions per surface element per sampled timestep. The bridge converts it to real solid mass removed over the coupling window with:

```
mass = reaction-count * sample-steps * fnum * solid-mass-per-reaction
```

and then divides by triangle area and the ablation timestep before passing the equivalent mass flux to the voxel ledger. This is the preferred path for chemically reacting DSMC ablation because SPARTA owns the surface reaction probability, species conversion, and reaction tallies.

`surface measure-flux` reads the same kind of DSMC reaction count data but does not apply mass loss. It sums the sampled reactions over the installed surface, converts them to a number flux with:

```
reaction-flux = sum(reaction-count) * fnum / (surface-area * DSMC-timestep)
```

and stores scalar diagnostics that can be checked later with `iac verify`. The `expected kinetic/theory` arguments compute the ideal-gas one-way impingement flux for a reactive species:

```
expected-reaction-flux =
  reaction-prob * mole-fraction * number-density
  * sqrt(kB*temperature/(2*pi*molecular-mass))
```

The command also stores `reaction-count-per-step` and `expected-reaction-count-per-step`, which are often the clearest DSMC sampling diagnostics. If `solid-mass-per-reaction` is supplied, it additionally stores `reaction-mass-flux` and `expected-reaction-mass-flux` in kg/m2/s by multiplying the number flux by the supplied solid mass consumed per reaction.

This command is the first DSMC-hosted regression hook for the coupled path: it tests geometry generation, surface installation, DSMC surface chemistry tallies, core diagnostics, and input-file verification without introducing voxel deletion or remeshing.

By default, DSMC-coupled sources advance voxel ablation by the elapsed DSMC time since the previous coupling update. The optional `ablation-dt` argument overrides that physical ablation time. For `dsmc/reaction`, `time-scale` multiplies both the sampled reaction-count mass and the ablation time. This is useful for quasi-steady chemistry probes: the input can sample a short DSMC window and advance the solid over a longer reservoir-equivalent ablation time. When comparing multiple DSMC sampling lengths at the same ablation time, choose `time-scale = ablation-update-time / (sample-steps * timestep)`.

The optional `mass-courant` argument chooses the ablation timestep from the sampled DSMC mass flux. The bridge maps the triangle mass fluxes through the ISTHMUS triangle-to-voxel ownership fractions, finds the active voxel with the smallest `remaining-mass / mapped-mass-rate`, and sets the timestep to `C` times that limiting time. For example, `mass-courant 1.0` advances to the next voxel-deletion event predicted by the current flux map, while `mass-courant 0.25` advances one quarter of the way to that event. `ablation-dt` and `mass-courant` are mutually exclusive.

The mass is stored on the triangle until a later command consumes it:

```
voxel ablate solid surface skin policy local delete yes
```

Selectors:

- `select all` applies flux to every triangle.
- `select normal` applies flux only when the triangle normal has

```
dot(normal, direction) >= min-cos.
```

- `select voxels` applies flux to triangles that have ISTHMUS ownership

entries for the named voxel model. This is a forward-compatible hook for voxel groups and material subsets.

`surface dump` writes VTP triangle files on scheduled run steps. The VTP cell data includes `area`, `requested-mass`, and `last-requested-mass`. The `last-requested-mass` field is useful for visual inspection because `requested-mass` is cleared after `voxel ablate` consumes it.

Inside DSMC, scheduled `surface dump` output is written when bridge commands advance the core step. Define the dump before the first surface generation so the active core model is initialized with the desired schedule.

`surface dump off` clears surface dumps that were already defined. Regression wrappers can use it after including a visual example input.

Inside DSMC, `surface write-vtp` writes a one-shot VTP snapshot immediately. This mirrors `voxel write-vtu`: DSMC owns the run loop, and the input script can write the current ISTHMUS surface after each `isthmus surface` regeneration. If the path does not contain `*`, the current core step number is inserted before the file extension.

The optional `fix <fix-id> fields <name1> ...` form reads per-surface values from a DSMC `fix ave/surf` and appends them as VTP triangle cell fields. The number of field names must match the number of per-surface columns exposed by the `fix`. This is the preferred visualization path for coupled DSMC runs because the ISTHMUS triangle geometry and DSMC surface quantities are written into the same VTP file.

15.4 Current Limits

- Only VTP surface dumps are implemented.
- Constant source flux and single-species kinetic-theory flux are implemented.

- DSMC `fix ave/surf` flux ingestion currently supports one MPI rank.
- `select voxels` currently distinguishes ownership presence for the current

voxel model, not separate voxel groups.

16 stats and stats_style Commands

The `stats` and `stats_style` commands control standalone console output.

16.1 Syntax

```
stats <N>
stats_style <column> <column> ...
```

16.2 Example

```
stats 1
stats_style step time active-voxels deleted-voxels remaining-mass mass-fraction volume-fraction front
```

16.3 Description

`stats N` prints every `N` steps and always prints step 0 and the final step.

`stats_style` selects columns from tracked history quantities. The standalone runner prints a title/configuration block first, then a fixed-width stats table.

16.4 Current Columns

```
step
time
active-voxels
deleted-voxels
remaining-mass
mass-fraction
volume-fraction
radius
front
requested-mass-step
applied-mass-step
dropped-mass-step
```

16.5 DSMC/SPARTA Note

In DSMC-hosted runs, use `iac stats` and `iac stats-style` for the core's own ablation table:

```
iac stats 1
iac stats-style step time active-voxels deleted-voxels remaining-mass mass-fraction radius
```

These commands do not change DSMC's native `stats` or `stats_style` settings. For coupled gas/solid runs, DSMC can continue printing particle/collision statistics while `iac stats` prints the voxel mass ledger after IAC ablation steps. Longer term, selected IAC quantities can also be exposed through DSMC compute/fix styles for native `c_...` or `f_...` output.

17 voxel dump Command

The `voxel dump` command writes voxel-model output.

Triangle surface dumps are configured with `surface dump`.

17.1 Syntax

```
voxel dump <id> <model> history <N> <path>
voxel dump <id> <model> vtu <N> <path> [select all|active] [scalar <field>]
voxel dump off
```

Also available in the DSMC bridge:

```
voxel dump <id> <model> history <N> <path>
voxel dump <id> <model> vtu <N> <path> [select all|active] [scalar <field>]
voxel write-vtu <model> <path> [select all|active] [scalar <field>]
```

17.2 Examples

```
voxel dump hist solid history 1 output/slab-direct-ablation/history.csv
voxel dump vox solid vtu 1 output/slab-direct-ablation/voxels_*.vtu select active scalar mass-fraction
voxel dump off
```

```
voxel write-vtu solid output/voxels-*.vtu select active scalar mass-fraction
```

17.3 Description

The `history` style writes one CSV summary after the run.

`voxel dump off` clears voxel dumps that were already defined. This is useful in regression wrappers that include a visual example input but should avoid writing output files during CTest.

The `vtu` style writes VTK unstructured-grid files during the run. A dump is written at step 0, every `N` steps, and at the final step. If the path contains `*`, that character is replaced by a zero-padded step number:

```
output/slab-direct-ablation/voxels_000004.vtu
```

If the path does not contain `*`, the step number is inserted before the file extension.

The `select` option controls which voxels are written:

- `select all` writes every voxel, including inactive/deleted voxels.
- `select active` writes only active voxels.

The default is `select active`, so depleted voxels disappear from VTU output when the ablation fix uses `delete yes`.

The `scalar` option sets the active VTK cell scalar. This is the field ParaView should choose first when the file opens. Supported values are:

```
mass-fraction
remaining-mass
active
fixed
id
ix
```

```
iy
iz
```

The default is `scalar mass-fraction`.

Inside DSMC, scheduled `voxel dump` output is written whenever a bridge command advances the core step, for example after `voxel ablate`. This is useful for no-gas verification and coupled loops that use explicit DSMC input-file commands for ablation updates.

`voxel write-vtu` writes a one-shot VTU snapshot immediately. This is useful in coupled loops where the input script decides exactly when a voxel snapshot should be written. It uses the same `select` and `scalar` options as scheduled `voxel dump ... vtu` output. If the path does not contain `*`, the current core step number is inserted before the file extension.

17.4 History Columns

```
step,time,active-voxels,deleted-voxels,remaining-mass,mass-fraction,
volume-fraction,front,radius,requested-mass-step,applied-mass-step,
dropped-mass-step
```

17.5 VTU Cell Data

The VTU dump writes hexahedral voxel cells with:

```
mass-fraction
remaining-mass
id
ix
iy
iz
active
fixed
```

Use `active` for thresholding deleted voxels and `mass-fraction` or `remaining-mass` for coloring ablation progress in ParaView.

17.6 Planned Extensions

- Restart dumps.
- Material/field output.

18 verify Command

The `verify` command compares tracked quantities to input-file exact/reference expressions.

18.1 Syntax

```
verify <quantity> exact "<expression>" tolerance <value> [absolute|percent] norm <final|max|rms>
convergence <quantity> exact "<expression>" tolerance <value> [absolute|percent] norm <final|max|rms> vary <variable>
```

18.2 Examples

```
verify remaining-mass exact "initial-mass - q1*area*time" tolerance 0.01 percent norm max
verify mass-fraction exact "1.0 - q1*time/(rho*length)" tolerance 0.01 percent norm max
verify volume-fraction exact "1.0 - q1*time/(rho*length)" tolerance 0.01 percent norm final
verify front exact "q1*time/rho" tolerance 0.01 percent norm final
verify radius exact "initial-radius - q1*time/rho" tolerance 3.0 percent norm final
convergence radius exact "initial-radius - q1*time/rho" tolerance 100.0 percent norm final vary resolution 5 10 20 va
```

18.3 Description

The command evaluates an exact expression against the selected tracked quantity. The executable exits with a nonzero status if the error exceeds the tolerance. This makes verification cases natural CTest tests.

Regression tests generally use percent tolerances. Add **percent** after the tolerance value to check:

```
100 * abs(actual - exact) / abs(exact)
```

Percent error is undefined when the exact value is zero unless the actual value is also zero.

Use **absolute** or omit the mode when an absolute tolerance is more appropriate.

18.4 Convergence

convergence reruns the same input file with variable overrides and checks the error trend. Variables are defined with:

```
variable resolution equal 20
variable steps equal 4
```

and referenced with `${resolution}` or `${steps}` elsewhere in the input.

Each **vary** clause must have the same number of values. The first **vary** variable is treated as the refinement variable for apparent-order calculation. For example, **vary resolution 5 10 20 vary steps 1 2 4** runs three cases:

```
resolution=5, steps=1
resolution=10, steps=2
resolution=20, steps=4
```

The convergence check requires monotone error reduction by default and checks that the apparent order from the first to last case is inside the requested **order** `<min>` `<max>` band.

18.5 Norms

`final` checks the last history row.

`max` checks the maximum absolute error over all recorded rows.

`rms` checks the root-mean-square error over all recorded rows.

18.6 Expression Variables

`time`
`t`
`step`
`dt`
`rho`
`density`
`dx`
`nx`
`ny`
`nz`
`length`
`area`
`initial-mass`
`voxel-mass`
`active-voxels`
`deleted-voxels`
`remaining-mass`
`mass-fraction`
`volume-fraction`
`voxel-volume-fraction`
`front`
`q1`
`source:q1`

19 run Command

The `run` command starts standalone time advancement.

19.1 Syntax

```
run <steps>  
run duration <time>
```

19.2 Examples

```
run 8  
run duration 0.02
```

19.3 Description

`run <steps>` advances a fixed number of steps.

`run duration <time>` advances for a requested physical duration using the current timestep. The model chooses enough whole steps to reach or slightly exceed the requested duration.

20 Direct Slab Ablation

This example removes exactly half of one voxel mass from each exposed slab column per step.

Input file:

```
units si

voxel material carbon density 1800.0
voxel create solid slab nx 8 ny 4 nz 4 dx 1.0e-6 material carbon

source q1 constant 1.8 units kg/m2/s
timestep mass/courant 0.5 source q1

# Compact callback-style alternative:
# fix ab all voxel/ablate 1 voxels solid source q1 policy local face xlo delete yes

stats 1
stats_style step time active-voxels deleted-voxels remaining-mass mass-fraction front

voxel dump hist solid history 1 output/slab-direct-ablation/history.csv
voxel dump vox solid vtu 1 output/slab-direct-ablation/voxels_*.vtu select active scalar mass-fraction

variable i loop 8
label ablate-loop
voxel ablate solid source q1 policy local face xlo delete yes
run 1
next i
jump SELF ablate-loop
```

Run it:

```
./build/ia-core -in examples/slab-direct-ablation/in.slab-direct-ablation
```

The example writes:

```
output/slab-direct-ablation/history.csv
output/slab-direct-ablation/voxels_000000.vtu
output/slab-direct-ablation/voxels_000001.vtu
...
output/slab-direct-ablation/voxels_000008.vtu
```

The VTU dump uses `select active`, so voxels removed by `delete yes` disappear from the visualized voxel mesh. It also uses `scalar mass-fraction`, so ParaView should open the files with `mass-fraction` as the active cell field.

The test form is:

```
include ../../../../examples/slab-direct-ablation/in.slab-direct-ablation

verify remaining-mass exact "initial-mass - q1*area*time" tolerance 0.01 percent norm max
verify mass-fraction exact "1.0 - q1*time/(rho*length)" tolerance 0.01 percent norm max
verify front exact "q1*time/rho" tolerance 0.01 percent norm final
```

Run the test:

```
ctest --test-dir build --output-on-failure
```

Print the stats table during the test:

```
cmake --build build --target check-verbose
```

Build the visual verification report:

```
cmake --build build --target test-report
```

The combined report is written to:

```
build/output/test-report.pdf
```

To report only this test, run:

```
python3 tools/run-test-report.py slab-direct-command-verification \  
  --out build/output/slab-direct-report.tex
```

21 ISTHMUS Slab Ablation

These examples ablate an 8 by 4 by 4 slab from the xlo ISTHMUS surface.

The finite-patch case intentionally exposes lateral edges:

```
examples/slab-isthmus-ablation/in.slab-isthmus-finite
```

It runs:

```
isthmus surface skin voxels solid buffer 1 weighting no map yes
surface flux skin source q1 select normal nx -1.0 ny 0.0 nz 0.0 min-cos 0.5
voxel ablate solid surface skin policy local delete yes
```

The ghost case adds y/z infinite-wall ghost voxels before surface generation:

```
voxel ghost solid axis y boundary infinite layers 1
voxel ghost solid axis z boundary infinite layers 1
isthmus surface skin voxels solid buffer 1 weighting no map yes crop real
```

Ghost voxels are surface-construction images only. They do not carry separate mass, and triangle ownership maps back to the corresponding real voxels. This removes finite-patch edge effects while keeping the ablation update on the real voxel domain.

Run either case:

```
build/ia-core -in examples/slab-isthmus-ablation/in.slab-isthmus-finite
build/ia-core -in examples/slab-isthmus-ablation/in.slab-isthmus-ghost
```

The regression wrappers are:

```
tests/inputs/slab-isthmus-ablation/in.slab-isthmus-finite-surface.verify
tests/inputs/slab-isthmus-ablation/in.slab-isthmus-ghost-wall.verify
```

The finite-patch wrapper uses broad tolerances because the exposed edges are expected to perturb the one-dimensional exact recession. The ghost wrapper uses tight tolerances because the infinite-wall construction should recover the one-dimensional slab solution.

22 Sphere ISTHMUS Ablation

This example reconstructs a triangle surface from a voxel sphere with ISTHMUS, applies a constant mass flux to all surface triangles, maps the triangle mass loss back to voxels, and deletes depleted voxels.

The current case uses a 1 mm diameter sphere with `resolution 20`, meaning 20 voxels across the diameter. It uses `timestep mass/courant 0.5 source q1`, which gives `dt = 0.05625 s` for the current material, grid spacing, and flux. The four-step test therefore runs to `0.225 s`.

22.1 Inputs

```
examples/sphere-isthmus-ablation/in.sphere-isthmus-local
examples/sphere-isthmus-ablation/in.sphere-isthmus-normal
```

The core loop is:

```
isthmus surface skin voxels solid buffer 1 weighting no map yes
surface flux skin source q1 select all
voxel ablate solid surface skin policy carryover/normal delete yes
run 1
```

This loop is intentionally command-driven. A future DSMC run can place DSMC convergence, flux mapping, voxel ablation, and surface regeneration in the same style of input-file loop.

22.2 Run

```
build/ia-core -in examples/sphere-isthmus-ablation/in.sphere-isthmus-local
build/ia-core -in examples/sphere-isthmus-ablation/in.sphere-isthmus-normal
```

22.3 Outputs

```
output/sphere-isthmus-local/
output/sphere-isthmus-normal/
```

Open the VTU files in ParaView to inspect active voxel mass fraction. Open the VTP files to inspect surface triangle area and the last applied surface mass request.

22.4 Verification

The automated wrappers are:

```
tests/inputs/sphere-isthmus-ablation/in.sphere-isthmus-local-deletion.verify
tests/inputs/sphere-isthmus-ablation/in.sphere-isthmus-normal-carryover.verify
```

It compares the inferred continuum-equivalent radius and mass fraction against the analytical shrinking-sphere solution:

```
radius = initial-radius - q1*time/rho
mass-fraction = (radius / initial-radius)^3
```

The local wrapper is the intentionally nonconservative reference path. It uses percent tolerances of `7.0 percent` for radius and `20.0 percent` for mass fraction at the final time.

The normal-carryover wrapper conserves mapped surface mass by pushing overshoot inward through live neighbor voxels. It uses tighter percent tolerances of `4.0 percent` for radius and `12.0 percent` for mass fraction, and it checks that final-step dropped mass is essentially zero.

23 DSMC Sphere Kinetic Example

This example runs from this repository using the private DSMC/IAC executable built by the overlay target. It is a smoke test for the embedded bridge: DSMC owns the gas domain, native `run` commands, particles, surface reactions, surface averaging, loops, `remove_surf`, and `surf_modify`; this core owns voxel state, ISTHMUS surface generation, DSMC-to-triangle reaction-count conversion, normal-carryover ablation, and regenerated surface geometry.

Run the chemistry case directly with:

```
cmake --preset dsmc
cmake --build --preset dsmc

cd examples/dsmc-sphere-kinetic
../../build-dsmc/bin/dsmc-iac \
  -screen none -log output/log.sparta -in in.dsmc-sphere-kinetic
```

To run the DSMC-enabled CTest suite:

```
ctest --preset dsmc
```

The normal standalone build does not include DSMC tests. A DSMC-enabled build runs the standalone tests plus the coupled DSMC chemistry test.

23.1 Chemistry Loop

The case uses a stationary initial O₂/N₂ gas with CO enabled as a product species. The convergence case uses outflow boundaries and face emission to approximate a fixed reservoir:

```
boundary      o o o
create_particles  air n 0 twopass
fix           reservoir emit/face air all twopass
surf_collide   1 diffuse 5000.0 1.0
```

The 5000 K diffuse surface is a verification choice: it keeps the wall at the same temperature as the reservoir so the analytical comparison is not polluted by cold-wall cooling. The separate visualization case still uses a 300 K wall to show the current gas-field behavior.

SPARTA handles the surface reaction:

```
species       air.species O2 N2 CO
mixture       air O2 frac 0.21
mixture       air N2 frac 0.79
mixture       air CO frac 0.0
surf_react    ox prob carbon-co.surf
surf_modify   all collide 1 react ox
compute       rco react/surf all ox
fix           rco ave/surf all 1 20 20 c_rco[*] ave one
```

The local `carbon-co.surf` file represents carbon monoxide formation as:

```
O2 --> CO + CO
D S 1.0
```

After each DSMC run segment, the bridge maps the averaged surface reaction counts to ISTHMUS triangles and removes two carbon atoms of solid mass per O₂ reaction:

```

surface flux skin dsmc/reaction fix rco column 1 sample-steps 20 \
  solid-mass-per-reaction 3.98894696e-26 time-scale 500
voxel ablate solid surface skin policy carryover/normal delete yes

```

Then native DSMC commands clear the old collision surface and install the regenerated ISTHMUS surface directly from memory:

```

remove_surf all
isthmus surface skin voxels solid buffer 1 weighting no map yes
surface install skin particle none type 1
surf_modify all collide 1 react ox

```

23.2 Analytical Comparison

The report compares the voxel history to a continuum shrinking-sphere solution for a uniform stationary O₂ reservoir:

```

Gamma-O2 = n-O2 * sqrt(kB*T/(2*pi*m-O2))
j = Gamma-O2 * reaction-probability * solid-mass-per-reaction
R(t) = R0 - j*t/rho-solid
mass-fraction = (R/R0)^3

```

That comparison is useful, but it has an important caveat. The reservoir boundary and hot verification wall make the case much closer to the analytical assumption than the earlier closed periodic box. It is still a finite DSMC domain with a finite emitted reservoir, so local O₂ depletion, CO production, and surface sampling noise can remain. The report should be read as a chemistry-coupling verification against the continuum target, not as a perfect monotone proof of the continuum limit.

Build the PDF/CSV report with:

```
cmake --build build-dsmc --target dsmc-convergence-report
```

or run the generator directly:

```

python3 tools/run-dsmc-kinetic-convergence.py \
  --dsmc build-dsmc/bin/dsmc-iac \
  --steps 5,20,80 \
  --loops 6 \
  --resolution 10 \
  --ablation-update-time 0.001 \
  --fnum 2.0e12 \
  --tolerance-percent 5 \
  --pdf

```

The runner keeps the ablation time per coupled update fixed. For each DSMC sampling length, it sets:

```
time-scale = ablation-update-time / (sample-steps * timestep)
```

so all cases advance the same solid ablation time while sampling the chemistry for different numbers of DSMC steps. The generated report plots remaining mass fraction and equivalent radius recession against the analytical curve.

For a deliberately deep exploratory probe, run:

```
cmake --build build-dsmc --target dsmc-half-mass-report
```

This target uses a coarser 5-voxel sphere and a large scaled ablation interval. It is intended for stress-testing the coupled plumbing, not for tight verification.

23.3 Visualization Case

The repository also includes a DSMC visualization input:

```
cd examples/dsmc-sphere-visual
../../build-dsmc/bin/dsmc-iac \
  -screen none -log output/log.sparta -in in.dsmc-sphere-visual
python3 ../../tools/convert-dsmc-visual.py .
```

or, from a DSMC-enabled CMake build:

```
cmake --build build-dsmc --target dsmc-visual-case
```

This case writes SPARTA-native fluid text dumps, core voxel/surface VTK files, and converted ParaView-ready gas files:

```
examples/dsmc-sphere-visual/output/grid.*.dump
examples/dsmc-sphere-visual/output/fluid-*.vtu
examples/dsmc-sphere-visual/output/voxels-*.vtu
examples/dsmc-sphere-visual/output/surface-*.vtp
examples/dsmc-sphere-visual/output/history.csv
```

Open `fluid-*.vtu`, `voxels-*.vtu`, and `surface-*.vtp` directly in ParaView. The `.dump` files are native SPARTA text dumps retained as converter inputs and diagnostics.

The grid dump uses:

```
compute gas thermal/grid all air temp press
compute frac grid all species massfrac
fix gasave ave/grid all 1 20 20 c_gas[*] c_frac[*] ave one
dump dgrid grid all 20 output/grid.*.dump id xc yc zc vol f_gasave[*]
```

The converted fluid VTU files include temperature, pressure, and O₂/N₂/CO mass fractions. The surface VTP snapshots use:

```
compute rco react/surf all ox
compute sqty surf all air n nflux_incident press
fix sqty ave/surf all 1 20 20 c_sqty[*] c_rco[*] ave one
surface write-vtp skin output/surface-*.vtp \
  fix sqty fields collision-count incident-number-flux pressure co-reaction-count
```

Those VTP files include the current ISTHMUS triangle surface, area and mass-request fields, DSMC collision count, incident number flux, surface pressure, and the averaged CO-forming reaction count used to remove carbon. The voxel VTU files write active voxels with `mass-fraction` as the active cell scalar.

Reaction heat release is not yet coupled into the gas. The standard diffuse wall collision model still sets product velocities from the 300 K wall temperature, so O₂ depletion and CO production are physically represented by species conversion, but an exothermic near-surface temperature rise is not yet part of this bridge.

24 Directory Layout

`include/isthmus_ablation/`
Public C++ headers for the core library.

`src/`
Core implementation, parser, expression evaluator, and standalone CLI.

`examples/`
Human-readable standalone input examples, organized as `<case>/in.<case>`.

`dsmc-bridge/`
DSMC command-style bridge sources. The dsmc CMake target symlinks these into `build-dsmc/dsmc-overlay/src`; they should not be copied into the DSMC checkout.

`tests/inputs/`
Regression wrappers, organized as `<case>/in.<case>.verify`.

`docs/`
Manual, command reference, design notes, and generated diagram/PDF artifacts.

`tools/`
Local documentation and development utilities.

`build-dsmc/dsmc-overlay/`
Generated private DSMC source overlay. This is disposable build output.

Generated files are ignored:

`build/`
`build-dsmc/`
`output/`
`site/`

The generated single-file manual is:

`docs/isthmus-ablation-core-manual.pdf`

25 Build And Link

This project has two build modes:

- standalone `ia-core`, which runs only this repository's input commands;
- DSMC/IAC overlay, which builds a private DSMC executable with IAC commands.

The overlay is the recommended user workflow. It does not copy files into the user's DSMC checkout and it does not edit DSMC source files.

25.1 What Is Built

The core library is:

```
libisthmus_ablation_core.a
```

When ISTHMUS is found, that library links against:

```
libisthmus_cpp.a
```

The standalone executable is:

```
build/ia-core
```

The DSMC/IAC executable is:

```
build-dsmc/bin/dsmc-iac
```

25.2 Root Discovery

CMake looks for DSMC and ISTHMUS in this order:

- CMake cache variables:

```
cmake --preset dsmc -DDSMC_ROOT=/path/to/dsmc -DISTHMUS_ROOT=/path/to/isthmus
```

- Environment variables:

```
export DSMC_ROOT=/path/to/dsmc
export ISTHMUS_ROOT=/path/to/isthmus
```

- Common local paths:

```
../dsmc
~/dsmc
../isthmus/build
../isthmus/build-wsl
~/isthmus/build
~/isthmus/build-wsl
```

ISTHMUS_ROOT should point to an ISTHMUS checkout or install that contains a CMake package under `build/` or `build-wsl/`. Users can also set `ISTHMUS_CPP_DIR` or `isthmus_cpp_DIR` directly to the directory containing `isthmus_cppConfig.cmake`.

25.3 Recommended DSMC/IAC Build

One-time shell setup:

```
export DSMC_ROOT=$HOME/dsmc
export ISTHMUS_ROOT=$HOME/isthmus
```

Build and test:

```
cmake --preset dsmc
cmake --build --preset dsmc
ctest --preset dsmc
```

Run a coupled input:

```
build-dsmc/bin/dsmc-iac \
-in examples/dsmc-sphere-kinetic/in.dsmc-sphere-kinetic
```

25.4 DSMC Machine Target

The default DSMC make target is:

```
mac_mpi
```

Override it with:

```
cmake --preset dsmc -DDSMC_MACHINE=mpi
cmake --build --preset dsmc
```

The executable generated by DSMC inside the overlay is `spa_<machine>`, and this repository creates a stable symlink:

```
build-dsmc/bin/dsmc-iac -> build-dsmc/dsmc-overlay/src/spa_<machine>
```

25.5 How The Overlay Works

The `dsmc` target creates:

```
build-dsmc/
  bin/
    dsmc-iac
  dsmc-overlay/
    README.md
    src/
      symlinks to DSMC src files
      symlinks to dsmc-bridge/*.cpp and *.h
      generated Makefile.package
      generated Makefile.package.settings
      Obj_<machine>/
      spa_<machine>
```

The script that creates this is:

```
tools/create-dsmc-overlay.py
```

It symlinks the user's DSMC `src` directory into the overlay, excluding build artifacts and generated style headers. It then symlinks these IAC bridge files:

```
dsmc-bridge/iac.cpp
dsmc-bridge/iac.h
dsmc-bridge/iacbridge.cpp
dsmc-bridge/iacbridge.h
dsmc-bridge/isthmus.cpp
dsmc-bridge/isthmus.h
dsmc-bridge/source.cpp
dsmc-bridge/source.h
dsmc-bridge/surface.cpp
dsmc-bridge/surface.h
dsmc-bridge/voxel.cpp
dsmc-bridge/voxel.h
```

DSMC's existing make system scans headers in the overlay and regenerates `style_command.h`. Because the bridge headers are present in the overlay, commands such as:

```
voxel ...
isthmus ...
surface ...
source ...
iac ...
```

become visible to DSMC without editing DSMC parser logic.

The overlay `Makefile.package` adds:

```
-I/path/to/isthmus-ablation-core/include
-I/path/to/isthmus/include
/path/to/isthmus-ablation-core/build-dsmc/libisthmus_ablation_core.a
/path/to/isthmus/build/libisthmus_cpp.a
```

or the equivalent paths discovered from the ISTHMUS CMake package.

25.6 Standalone Build

Standalone mode only needs this repository and, for ISTHMUS cases, ISTHMUS:

```
cmake --preset standalone
cmake --build --preset standalone
ctest --preset standalone
```

Run a standalone ISTHMUS example:

```
./build/ia-core -in examples/sphere-isthmus-ablation/in.sphere-isthmus-normal
```

If ISTHMUS is not found, CMake still builds direct voxel functionality, but ISTHMUS tests and examples are unavailable.

25.7 Debugging The Overlay

The overlay is disposable. To force a clean DSMC/IAC rebuild:

```
rm -rf build-dsmc/dsmc-overlay build-dsmc/bin/dsmc-iac
cmake --build --preset dsmc
```

To inspect the generated DSMC package settings:

```
cat build-dsmc/dsmc-overlay/src/Makefile.package
```

To bypass the overlay and run tests against an existing DSMC executable:

```
cmake -S . -B build-dsmc-external \
  -DIAC_DSMC_USE_OVERLAY=OFF \
  -DIAC_DSMC_EXECUTABLE=/path/to/spa_mac_mpi
cmake --build build-dsmc-external
ctest --test-dir build-dsmc-external --output-on-failure
```

That external path is mainly for debugging old builds. New users should prefer the overlay.

26 Testing

The project uses CTest.

Build and run tests:

```
cmake --preset standalone
cmake --build --preset standalone
ctest --preset standalone --output-on-failure
```

CTest captures output from passing tests by default. To see the stats table while the test runs, use verbose CTest output:

```
ctest --preset standalone --output-on-failure --verbose
```

or use the convenience target:

```
cmake --build --preset standalone --target check-verbose
```

To build the optional visual verification report:

```
cmake --build --preset report
```

This runs the tests, collects each configured test report CSV, and writes:

```
build/output/test-report.pdf
```

To select a subset directly:

```
python3 tools/run-test-report.py all
python3 tools/run-test-report.py 1-3
python3 tools/run-test-report.py slab-direct-command-verification
```

The standalone regression tests are:

```
slab-direct-command-verification
slab-direct-yhi-verification
slab-isthmus-finite-surface-verification
slab-isthmus-ghost-wall-verification
sphere-isthmus-local-deletion-verification
sphere-isthmus-normal-carryover-verification
sphere-isthmus-kinetic-theory-verification
sphere-isthmus-normal-carryover-convergence
```

These are standalone/core tests. They run with the `ia-core` binary and should be the only tests present in a normal standalone build.

To add coupled DSMC tests, configure and build the DSMC/IAC overlay:

```
cmake --preset dsmc
cmake --build --preset dsmc
ctest --preset dsmc
```

That build runs the standalone/core tests plus:

```
dsmc-slab-direct-ablation-verification
dsmc-sphere-isthmus-normal-carryover-verification
dsmc-sphere-flux-verification
```

The DSMC tests are intentionally fast. The flux verification is a one-step instantaneous kinetic-theory check: it verifies the initial O2 reaction count and equivalent carbon mass flux before the perfectly consuming surface depletes the nearby O2 population. Longer DSMC depletion and coupled-ablation studies remain available as examples and report targets, but they are not part of the automatic CTest suite.

To build the DSMC convergence report:

```
cmake --build --preset dsmc --target dsmc-convergence-report
```

It writes:

```
build-dsmc/output/dsmc-sphere-kinetic-convergence/summary.csv
build-dsmc/output/dsmc-sphere-kinetic-convergence/report.pdf
```

The standalone test report currently includes:

```
tests/inputs/slab-direct-ablation/in.slab-direct-command.verify
tests/inputs/slab-direct-ablation/in.slab-direct-yhi.verify
tests/inputs/slab-isthmus-ablation/in.slab-isthmus-finite-surface.verify
tests/inputs/slab-isthmus-ablation/in.slab-isthmus-ghost-wall.verify
tests/inputs/sphere-isthmus-ablation/in.sphere-isthmus-local-deletion.verify
tests/inputs/sphere-isthmus-ablation/in.sphere-isthmus-normal-carryover.verify
tests/inputs/sphere-isthmus-ablation/in.sphere-isthmus-kinetic-theory.verify
tests/inputs/sphere-isthmus-ablation/in.sphere-isthmus-normal-carryover-convergence.verify
```

The command-loop test includes the example case:

```
include ../../../../examples/slab-direct-ablation/in.slab-direct-ablation
```

The fix test keeps the compact callback-style path covered while the examples move toward explicit `voxel ablate` loops. Tests pass only if all `verify` commands in the wrapper input files pass.

The sphere ISTHMUS tests are enabled when the build finds the ISTHMUS C++ package. They run marching cubes, apply constant triangle flux, map that flux back to voxels, and compare the inferred radius and mass fraction against the continuum shrinking-sphere solution. The local sphere test keeps the nonconservative path covered. The normal-carryover sphere test checks the conservative path and verifies that final-step dropped mass is essentially zero.

The slab ISTHMUS tests compare a finite 4 by 4 surface patch against the same patch with y/z infinite-wall ghost voxels. The finite-patch case has broad tolerances because edge effects are expected. The ghost case should match the one-dimensional slab recession nearly exactly.

The convergence test is an input file, not an external runner. It uses `variable ... equal ...` and a `convergence ... vary ... order ...` command to run normal carryover at 5, 10, and 20 voxels across the diameter with mass-Courant timing. It requires monotone radius-error reduction. The voxelized volume-fraction check allows non-monotone coarse stair-step ties but still checks the end-to-end apparent order in a broad first-order band.

26.1 Test Organization

Use `tests/inputs/` for automated regression wrappers. Keep these compact and deterministic. Prefer including a file from `examples/` and adding only the `verify` commands or test-specific criteria in the wrapper.

Use **examples/** for readable user-facing examples. Examples should describe and run the physical case without embedding every regression criterion.

Example cases should use DSMC/SPARTA-style descriptive folders and input names:

```
examples/<case-name>/in.<case-name>  
tests/inputs/<case-name>/in.<case-name>.verify
```

Standalone convergence tests should live under **tests/inputs/** and use the explicit **convergence** verification command. Coupled DSMC convergence tests can use a runner under **tools/** when they need to launch DSMC repeatedly and generate per-case input files under **build/output**.

27 Test Reports

Verification reports are optional visual artifacts for inspecting regression cases. They are not part of the core physics model.

27.1 Command-Line Report Output

The executable can write verification data when the test runner asks for it:

```
build/ia-core -in tests/inputs/slab-direct-ablation/in.slab-direct-command.verify \
  -report-csv build/output/test-report-data/slab-direct-command-verification/report.csv
```

The CSV contains:

```
quantity,expression,step,time,actual,exact,error,tolerance,tolerance-mode,norm,pass
```

This data-first design is intentional. Future DSMC/SPARTA-coupled runs can emit the same verification/report CSV without depending on standalone plotting code or modifying the input file.

27.2 Report Cases

Reportable tests are listed in:

```
tests/report-cases.csv
```

Each row gives the report index, test name, input file, and optional requirements. Add future regression cases there when they should be available to the report runner.

```
index,name,input,requires,expected-fail
2,slab-direct-yhi-verification,tests/inputs/slab-direct-ablation/in.slab-direct-yhi.verify,,no
3,slab-isthmus-finite-surface-verification,tests/inputs/slab-isthmus-ablation/in.slab-isthmus-finite-surface.verify,i
4,slab-isthmus-ghost-wall-verification,tests/inputs/slab-isthmus-ablation/in.slab-isthmus-ghost-wall.verify,isthmus,n
5,sphere-isthmus-local-deletion-verification,tests/inputs/sphere-isthmus-ablation/in.sphere-isthmus-local-deletion.ve
6,sphere-isthmus-normal-carryover-verification,tests/inputs/sphere-isthmus-ablation/in.sphere-isthmus-normal-carryove
7,sphere-isthmus-kinetic-theory-verification,tests/inputs/sphere-isthmus-ablation/in.sphere-isthmus-kinetic-theory.ve
8,sphere-isthmus-normal-carryover-convergence,tests/inputs/sphere-isthmus-ablation/in.sphere-isthmus-normal-carryover
```

The runner auto-detects optional features from the CMake build cache. Cases with unmet requirements are skipped when selecting `all`. Cases marked `expected-fail` are allowed to return a verification failure, and their CSV data is still included in the generated report.

27.3 Build The Combined Report

Run all report cases and build one combined PDF:

```
cmake --build build --target test-report
```

The generated files are:

```
build/output/test-report.tex
build/output/test-report.pdf
build/output/test-report-data/<test-name>/report.csv
```

The report includes:

- a table of contents for jumping to individual cases and input listings;
- a global summary table with case name, quantity, error, tolerance, norm, and pass/fail;
- one section per test case, starting on a new page;
- the test input listing for each case;
- any input files included by the test input, also starting on new pages;
- actual vs exact plots for each verified quantity;
- error plots with tolerance lines for each verified quantity;
- convergence order tables for convergence cases;
- convergence plots with each grid resolution shown in the legend.

27.4 Direct Tool Use

The report runner accepts the same selection styles we expect to use as the suite grows:

```
python3 tools/run-test-report.py all
python3 tools/run-test-report.py 1-4
python3 tools/run-test-report.py slab-direct-command-verification
python3 tools/run-test-report.py slab-direct-command-verification,slab-isthmus-ghost-wall-verification
```

Optional features can be overridden explicitly:

```
python3 tools/run-test-report.py all --available isthmus
python3 tools/run-test-report.py all --available ""
```

Names can be shortened if the partial name matches exactly one case:

```
python3 tools/run-test-report.py fix-verification \
--out build/output/fix-report.tex
```

The lower-level report builder can still aggregate already-written CSV files:

```
python3 tools/build-test-report.py --discover build/output/test-report-data \
--out build/output/discovered-report.tex --pdf
```

Discovery mode does not know input files unless named cases are provided, so the normal workflow uses `tools/run-test-report.py`.

27.5 Convergence Reports

When a test input uses the `convergence` command, the executable writes extra CSV files next to the normal `report.csv`:

```
convergence.csv
convergence-order.csv
convergence-<quantity>-<case>.csv
```

The report generator uses these files to add a convergence order table and plots. The table records the first and last refinement values, the corresponding errors, the measured order, the allowed order band, whether monotone error reduction was required, and the pass/fail result. The plots show the convergence error points and actual/exact time histories for each resolution with legend entries identifying the varied input values.

28 Documentation

Documentation is part of the development loop. Whenever a command or behavior is added or edited, update the relevant Markdown page in `docs/` in the same change.

28.1 Source Format

The manual source is plain Markdown. Command pages live in:

```
docs/commands/
```

Concept pages live in:

```
docs/concepts/
```

Examples live in:

```
docs/examples/
```

Development notes live in:

```
docs/development/
```

The web documentation layout is described by:

```
mkdocs.yml
```

The intended web-docs stack is MkDocs Material because it is modern, searchable, widely used, and still keeps every page as readable Markdown.

28.2 PDF Manual

The repository also has a local PDF builder:

```
cmake --preset standalone  
cmake --build --preset docs
```

It writes:

```
docs/isthmus-ablation-core-manual.pdf
```

The lower-level builder is:

```
python3 tools/build-docs-pdf.py
```

This PDF builder is intentionally lightweight. It is a dependable review artifact for local development, while MkDocs Material remains the richer web documentation path.

28.3 Update Rule

When functionality changes:

- update the command reference for syntax changes;
- update concepts if the architecture or verification model changes;
- update examples when user-facing input files change;
- rebuild the PDF manual before handing the change back.